

Exact decision policies

Operating regions, collision geometry and adaptive diagnostics

Complete bounded calibration regions through vertex-oracle discovery, distinct cubic and monotone polynomial images, and globally optimal resettable diagnostic programs.

15

coupled operating cells

231

lazy-window feasible settings

3

optimal diagnostic cost

Exact reusable policies on declared bounded models. Complete cubic collision geometry; positive-cost resettable minimax diagnostics. Python evidence, no new Lean theorem or physical derivation.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

From individual answers to reusable decisions

This release implements the three proposed machinery extensions: complete calibration decision regions, broader distinct-value projection through collision geometry, and globally optimal resettable diagnostic programs. Ten persistent catalogue kinds now include `calibration_policy` and `diagnostic_policy`. The new objects compile once and answer subsequent decisions from their stored exact geometry or tree.

The direct output for a calibration engineer is an operating policy: every integer setting that can win on a chosen target region, its switching boundaries, and every tie. The output for a dataset or configuration author is an addressable distinct image for an integer cubic, or a higher-degree field with a proved discrete monotonicity condition. The output for a diagnostics author is an executable adaptive program that identifies an entire supplied hypothesis set with minimum worst-case cost. No new compatible-tile benchmark is included.

The integrated example has fifteen nonnegative integer settings with three components summing to four. Its target policy has fifteen cells and thirty nonempty pair contacts. Forty-two complete optimizer queries in three discovery rounds establish the policy. An independent enumeration checks 289 rational target points. The same fifteen settings become hypotheses for a diagnostic program whose worst-case cost is three, compared with four for the cheapest single complete readout. A narrow operating window discovers just one winning setting among 231 feasible settings using four optimizer queries, without building a candidate list.

The exact calibration geometry

Fix a finite feasible set F of integer settings, defined by the supplied box, observation equations and linear inequalities. Fix a positive definite rational metric M . The cost of setting v at target t is $(v-t)^T M (v-t)$. Its comparison with w is $v^T M v - w^T M w - 2(v-w)^T M t$: the quadratic target term cancels exactly.

Parameterize a target plane or line as $t=o+B s$, where o and B are rational and B has full column rank. The supported parameter dimension is one or two. The user supplies a closed positive-width interval or rectangle in s coordinates. Each setting's winning region is the intersection of this box with the affine halfspaces saying its cost is no greater than every competitor's cost. The regions are closed. Their intersections preserve ties, including segments, isolated points and settings tied throughout a whole region.

In explicit form, the inequality for v against w has normal $-2 B^T M (v-w)$ and right side $w^T M w - v^T M v + 2(v-w)^T M o$. The stored packet retains each normal, bound and competitor. All clipping and intersection coordinates are rational. The implementation handles polygons, line segments and singletons, rather than dropping a setting merely because its cell has zero area.

The policy evaluator checks these stored halfspaces at the requested rational parameter. It returns every winning setting and their common exact energy. It does not run the integer optimizer again. Parameters outside the compiled box are rejected. If the fixed

feasible set is empty, the policy explicitly reports that condition and returns no settings.

Discovering the policy without listing all settings

CalibrationPolicy initializes its candidate set from complete bounded-optimizer calls at the corners of the parameter box. It constructs the provisional cells of those candidates. It then asks the same optimizer for all minimizers at every provisional cell vertex. New winners are inserted and the cells rebuilt. Cached vertex queries avoid repeated solves. The process stops when a complete round discovers no new setting.

The completeness argument is finite and exact. On any final cell belonging to v , the difference between an arbitrary feasible competitor w and v is affine in s . At every cell vertex, the complete optimizer confirms that v is globally optimal, so that difference is nonnegative there. An affine function nonnegative at every vertex of a compact convex polytope is nonnegative throughout it. Thus no undiscovered setting can beat v anywhere in the cell. The provisional cells cover the parameter box because a finite nonempty candidate set always has a minimum there.

Ties are covered by the same argument. If an undiscovered setting ties v on a face, the nonnegative affine difference vanishes on that face and at one or more vertices. Since each vertex query returns all minimizing settings, the tied setting would have been discovered. This includes boundary-only cells and globally coincident costs on the target slice. It is why an optimizer returning just one tied setting would be insufficient for this compiler.

The implementation supports the declared finite model, not an unbounded parametric mixed-integer optimization language. Total optimizer nodes, query count and discovered-setting count have explicit budgets. A budget failure returns no completed policy. The complete output may itself be large if many settings have nonempty regions. Optimizer discovery reduces the need to materialize irrelevant settings; it cannot compress away genuinely distinct required regions.

Switching boundaries and the offline workbench

The packet contains every nonempty pairwise cell intersection as a contact, with exact vertices. Each contact describes where the two settings are simultaneously optimal. An intersection may be a point, an edge or a higher-dimensional coincident-cost region. These are exact contacts within the declared target box, not an assertion about neighboring cells outside it.

The offline `policy_workbench.html` displays the compiled cells as SVG geometry and accepts integer, decimal or fractional target parameters. Geometry coordinates are converted to ordinary numbers only for drawing. Halfspace decisions use BigInt rational arithmetic. Setting labels and results preserve integer digits as strings. An executable client check exercises an exact one-third triple tie, a decimal with more than 53 bits of information, and rejection outside the compiled region. Native browser layout testing is not claimed.

Boundary-specific tests compile a rectangle containing four point cells, four segment cells and one area cell. Another test has distinct settings tied permanently across a target plane. Random rational targets and off-diagonal metrics are compared against independently exhausted finite setting sets. These checks exercise features that a sampled colour map alone cannot establish.

Complete cubic collision geometry

For the integer cubic $f(n)=a n^3+b n^2+c n+d$, with a nonzero, an off-diagonal collision factors as $f(x)-f(y)=(x-y)[a(x^2+xy+y^2)+b(x+y)+c]$. Set $s=x+y$ and $h=y-x$. Completing the square gives $(3a s+2b)^2+3a^2h^2=4b^2-12ac$. The constant d cancels.

The right side R is an exact integer. If R is negative, there are no off-diagonal real or integer collisions. Otherwise every distinct integer collision with $x<y$ has $1\leq h\leq\text{floor}(\sqrt{R/(3a^2)})$. For each h , the remaining square $R-3a^2h^2$ must be an integer square u^2 . Both signs of u are checked. The congruence $u-2b$ divisible by $3a$ gives s , and the parity $s-h$ even gives integer $x=(s-h)/2$ and $y=(s+h)/2$. Every retained pair is replayed through the original polynomial.

This is a complete finite collision procedure for integer cubics whenever the explicit ellipse enumeration fits its work budget. The bound depends on the coefficients, not the cardinality of the source population. Large ellipse bounds may exceed the budget; such a failure does not produce a complete projection. Translation can leave the bound small even when all relevant roots have enormous coordinates.

For $f(n)=n^3-n$, the off-diagonal pairs are $(-1,0)$, $(-1,1)$ and $(0,1)$: all three parameters map to zero. Over n in $[-10^{50},10^{50}]$, the source has $2\times10^{50}+1$ identities and the distinct image has $2\times10^{50}-1$ values. The canonical owner of zero is -1 and its multiplicity is three. A translated test moves this collision cluster to coordinate 10^{50} without scanning the enormous domain.

For arbitrary source predicates, only collision pairs with both endpoints in the source contribute duplicate ownership. The larger endpoint is excluded. Since all pairs of any equal-value cluster are present, this retains exactly its least valid source parameter even when modular restrictions remove other roots. Rank, select, locate, sample and shard operations then reuse the existing representative-domain machinery. Locate and multiplicity still solve complete original integer fibres; value identity and source-parameter identity remain different contracts.

Higher-degree monotone projections

For degrees above three, this release also recognizes strict discrete monotonicity on the convex integer hull of the finite source domain. It constructs $\Delta f(n)=f(n+1)-f(n)$. Complete sign-domain queries count violations of $\Delta f(n)>0$ and of $\Delta f(n)<0$ over every integer step between the smallest and largest source parameters. Zero violations in either direction establish injectivity on that hull, and hence on the possibly disconnected or modular source subset.

The packet retains the difference polynomial, hull, direction and violating counts. The example $f(n)=n^5+n$ over $\pm 10^{50}$ preserves all $2 \times 10^{50}+1$ values. No duplicate catalogue or value enumeration is needed. The method is conservative: a field may be injective on a sparse source without being strictly monotone throughout its convex hull. Nonmonotone higher-degree cases still require a complete collision backend and are rejected here. The existing constant, linear and quadratic reflection routes remain available.

Independent collision tests exhaust 308 cubic coefficient combinations against all pairs in a containing integer box. Additional tests vary source predicates, verify least owners and multiplicities, and exercise large translations, empty or singleton sources, work limits and unsupported higher-degree fields.

Globally optimal diagnostic programs

DiagnosticPolicy accepts named finite hypotheses, named linear readouts, a list of linear operator contexts and their exact costs. At most sixteen hypotheses are supported. Each experiment resets the model to its original unknown hypothesis, applies a chronological operator word and reads one output. Readouts are noiseless, reset has zero cost, operator costs are strictly positive, and readout costs are nonnegative. These semantics are necessary parts of the optimization problem.

The compiler constructs the observable row span over the whole supplied state space, including all future operator words. This extends beyond the original seed-reachable machine: a hypothesis is not excluded merely because a chosen seed cannot reach it. Independent witness rows are retained, and closure expresses every transformed row and supplied readout in the same exact observable coordinates. Hypotheses with identical observable coordinates have identical futures. If any such pair exists, the result names the indistinguishable groups rather than claiming individual diagnosis.

Finite witness words provide a valid initial adaptive tree and a finite worst-case upper bound U . To discover all potentially useful experiments, Dijkstra search enumerates operator words whose operation cost plus a readout can be at most U . It tracks the transformed observable differences between hypotheses, removing their common state. Equal relative-state tuples reached more expensively are dominated: every future experiment has the same partition from either tuple. Readouts partition the full hypothesis set by exact predicted value. Only the cheapest experiment for each partition is retained.

This cost restriction is sufficient globally. An optimal program can cost no more than the available incumbent U , and no experiment on one of its realized paths can cost more than that whole path. Therefore every experiment needed by an optimal tree appears in the bounded partition search. Positive operator costs bound relevant word lengths. State, partition, bit and dynamic-programming budgets remain explicit; exhaustion returns no completed optimum.

The exact minimax recursion and execution

For a remaining hypothesis subset S , define $V(S)=0$ when it contains one hypothesis. Otherwise, for each experiment that splits S into proper nonempty branches, compute its cost plus the largest V among those branches. The minimum is $V(S)$. Memoization reuses repeated subsets; every branch strictly reduces the subset, so free readouts do not create recursive cycles. One minimizing program is returned.

The packet stores node identities, remaining hypotheses, chosen word/readout, exact cost, predicted branch values and children. `step(node,value)` advances after a real readout consistent with the declared model. `run(hypothesis)` executes the full program on a supplied model hypothesis, resetting before each experiment, and records the path and accumulated cost. Unexpected readouts are rejected. The tree is a policy for this supplied finite noiseless model, not a statistical diagnosis or a nonresettable experiment controller.

Readout costs may be supplied by name, which is recommended. Positional costs follow canonical sorted readout names; operators retain list order. Canonical ordering makes persistence independent of JSON object insertion order. The integrated fifteen-setting example uses costs one for the first component, two for the second, eight for the third, and four for a combined complete readout. Its adaptive optimum is three. Independent tests enumerate all affordable chronological words and solve a separate subset recurrence for eight small multi-operator models, then compare the resulting costs and every realized path.

Persistence, reproduction and scope

Both policy kinds are persistent catalogue objects with method allowlists. Calibration exposes summary, evidence and decide; diagnostics exposes summary, evidence, step and run. `InverseDesign.policy` and `SequenceLibrary.diagnostic` also provide direct compilation methods. The service transcript includes both new policies and cubic projection, while SQLite restart tests exercise persisted decisions.

```
export PYTHONPATH=python
python python/develop_decision_policies.py --output /tmp/pp-policies
python -m unittest discover -s python/tests -p test_decision_policies.py
python -m perfectpower service --database /tmp/pp-policies.sqlite \
  < receipts/decision_policies/service_requests.jsonl
make test
```

The complete source ZIP retains previous research and receipts without an archive cap, including the separate invariant-interaction work described in `docs/VALENTINER_INVARIANT_INTERACTIONS.md`. The accompanying monograph describes the exact constructive arguments and model boundaries of the three decision-machinery extensions. These policy algorithms add no physical derivation or Lean theorem and do not extend arbitrary smooth global geometry. The retained physics work has its own specified mediator assumptions, exact invariant receipts and numerical alignment checks; those claims must be assessed against that evidence.